Отладка и инструменты разработчика

Отладка и инструменты разработчика

Игра тормозит. Враг ведёт себя странно. Коллизия не срабатывает. Как понять, что происходит, если всё летит со скоростью 60 кадров в секунду?

Стандартные профайлеры выдают стену чисел - непонятно, куда смотреть. Отладчик с брейкпоинтом останавливает всю игру - бесполезно для реалтайма. Решение: **встроить инструменты отладки прямо в игру**.

Визуальная отладка

Самый мощный инструмент - **рисовать то, что обычно невидимо**. Хитбоксы, радиусы обнаружения, пути ИИ, состояния врагов - всё это существует в коде, но не отображается. Сделаем так, чтобы отображалось.

Одна клавиша - и вы видите всё, что раньше было скрыто. Враг не атакует? Посмотрите на его состояние - может, он застрял в Patrol. Коллизия не срабатывает? Хитбоксы не пересекаются - видно сразу. Путь A* ведёт в стену? Нарисуйте его.

Рисуйте отладку **поверх** всего, после основного `draw`. И оборачивайте в `if debug.enabled` - чтобы в релизе это не отрисовывалось.

Цветовое кодирование

Ещё нагляднее - красить врагов по состоянию:

Python C#

```python
from enum import Enum, auto

class AIState(Enum):
    PATROL = auto()
    CHASE = auto()
    ATTACK = auto()
    FLEE = auto()

STATE_COLORS = {
    AIState.PATROL: (0, 255, 0),
    AIState.CHASE:  (255, 255, 0),
```

```
    AIState.ATTACK: (255, 0, 0),
    AIState.FLEE:   (0, 100, 255),
}

color = STATE_COLORS.get(entity.state, (255, 255, 255))
pygame.draw.rect(screen, color, rect, 2)
```

Смотрите на экран - сразу видно, кто что делает. Не нужно никаких логов.

Замедление и покадровый шаг

Хм. Баг проявляется на долю секунды и исчезает. Как его поймать? Замедлить время.

Python C#

```
time_scale = 1.0  # normal speed
paused = False
step_one = False

# input handling
if keys[pygame.K_F2]:
    time_scale = 0.1  # slow motion
if keys[pygame.K_F3]:
    time_scale = 1.0  # normal
if keys[pygame.K_F4] and not prev_keys[pygame.K_F4]:
    paused = not paused
if keys[pygame.K_F5] and not prev_keys[pygame.K_F5]:
    step_one = True  # advance one frame

# game loop
if not paused or step_one:
    effective_dt = delta_time * time_scale
    update_all(world, effective_dt)
    step_one = False
draw_all(screen, world)
```

При `time_scale = 0.1` всё замедляется в 10 раз. Анимации, ИИ, частицы, физика - всё, что зависит от deltaTime. Баг с коллизией, который мелькал на долю секунды, теперь длится несколько секунд и виден в деталях.

Покадровый шаг (`F4 + F5`) ещё мощнее: остановили игру, нажали F5 - прошёл один кадр. Ещё раз - ещё один кадр. Можно буквально рассматривать каждый шаг обновления.
Но имейте в виду: некоторые баги могу пропасть или проявиться на меньшем FPS.

Счётчик FPS и время кадра

Ок. Базовый, но незаменимый инструмент - видеть, сколько времени занимает каждый кадр.

Python C#

```python
class FPSCounter:
    def __init__(self, size=120):
        self.size = size
        self.frame_times = [0.0] * size  # circular buffer
        self.index = 0

    def update(self, delta_time):
        self.frame_times[self.index] = delta_time
        self.index = (self.index + 1) % self.size

    def get_fps(self):
        avg = sum(self.frame_times) / self.size
        return 1.0 / avg if avg > 0 else 0.0

    def get_avg_ms(self):
        return sum(self.frame_times) / self.size * 1000

    def draw_graph(self, screen, x, y, width, height):
        # draw frame time history as a bar graph
        bar_width = width / self.size
        target_ms = 16.67  # 60 FPS target

        for i in range(self.size):
            idx = (self.index + i) % self.size
            ms = self.frame_times[idx] * 1000
            bar_height = min(ms / target_ms * (height / 2), height)
            color = (0, 255, 0) if ms < target_ms else (255, 0, 0)
            bx = int(x + i * bar_width)
            pygame.draw.rect(screen, color,
                             (bx, int(y + height - bar_height),
                              max(int(bar_width), 1), int(bar_height)))

        # 16.67ms line (60 FPS target)
        line_y = int(y + height / 2)
        pygame.draw.line(screen, (255, 255, 0), (x, line_y), (x + width, l:


fps = FPSCounter()

# in game loop:
```

```
    fps.update(delta_time)
    fps.draw_graph(screen, 10, 10, 200, 60)
```

Зелёные столбики - кадр уложился в 16.67 мс (60 FPS). Красные - не уложился. Жёлтая линия - граница. Видите красный всплеск? Что-то в этом кадре заняло слишком много времени.

Обратите внимание: `frame_times` - это **кольцевой буфер** (circular buffer). Т.е. фиксированный массив, индекс ходит по кругу. Не растёт, не аллоцирует память. Это важно - инструмент отладки не должен сам создавать проблемы с производительностью.

Кольцевой буфер - полезная структура данных не только для FPS. Она используется для истории ввода (input buffer), реплея, буферов сетевых пакетов, аудиосэмплов.

Профилирование по системам

FPS-счётчик показывает **что** тормозит (конкретный кадр). Но не показывает **где**. Для этого нужно замерять время каждой подсистемы отдельно.

Python | C#

```python
import time


class Profiler:
    def __init__(self):
        self.timings = {}  # {"Physics": 2.1, "AI": 5.3, ...}
        self._start = 0

    def begin(self, name):
        self._start = time.perf_counter()

    def end(self, name):
        elapsed = (time.perf_counter() - self._start) * 1000
        self.timings[name] = elapsed

    def draw(self, screen, font, x, y):
        total = sum(self.timings.values())
        line_y = y
        for name, ms in self.timings.items():
            pct = ms / total * 100 if total > 0 else 0
            text = f"{name}: {ms:.1f}ms ({pct:.0f}%)"
            surface = font.render(text, True, (255, 255, 255))
            screen.blit(surface, (x, line_y))
            line_y += 18
```

```
25
26
27 profiler = Profiler()
28
29 # game loop
30 profiler.begin("Physics")
31 physics_system(world, dt)
32 profiler.end("Physics")
33
34 profiler.begin("AI")
35 ai_system(world, dt)
36 profiler.end("AI")
37
38 profiler.begin("Render")
39 render_system(world, screen)
40 profiler.end("Render")
41
42 profiler.draw(screen, font, 10, 80)
```

Теперь видно: "Physics: 1.2ms (8%), AI: 8.5ms (55%), Render: 5.7ms (37%)". ИИ жрёт больше половины кадра - оптимизировать нужно именно его.

Это гораздо полезнее, чем абстрактное "игра тормозит". Вы видите **конкретную систему** и можете разбираться прицельно.

Big-O на практике

Профайлер по системам - это ещё и способ **увидеть** алгоритмическую сложность.

Допустим, ваша проверка коллизий сравнивает каждый объект с каждым - $O(n^2)$. При 10 объектах это 100 проверок - незаметно. При 50 - 2 500. При 200 - 40 000.

Эксперимент: добавляйте объекты и смотрите на профайлер.

Получается, время растёт **квадратично** - это O(n²) прямо на экране. Переключили на [пространственное разбиение] - и вдруг 200 объектов обрабатываются за 1.1ms вместо 28.5ms.

Вы только что **измерили** разницу между O(n²) и O(n). Не в теории на лекции, а в своей игре, в реальном времени.

Добавьте кнопку, которая спавнит 50 объектов. Нажимайте и наблюдайте, как растёт время в профайлере. Это самый наглядный способ понять Big-O.

Естественно, все это относительно условно, и не всегда сразу или идеально точно вы увидите простой и понятный график. В виду того, что такое Big-O.

Умное логирование

Ну, `print("here")` работает, но быстро превращается в кашу. Несколько правил, которые сделают логи полезными:

Python C#

```python
import time

LOG_AI = True
LOG_PHYSICS = False
LOG_PERF = True


```

```
def log(category, enabled, message):
    if not enabled:
        return
    timestamp = time.perf_counter()
    print(f"[{timestamp:.3f}] [{category}] {message}")


# usage:
log("AI", LOG_AI, f"Enemy #{eid} state: {state} -> AIState.CHASE")
log("PERF", LOG_PERF, f"Frame took {dt*1000:.1f}ms")

# conditional: only log when something goes wrong
if delta_time > 0.033:  # below 30 FPS
    log("PERF", True, f"SPIKE: {delta_time*1000:.1f}ms")
```

Ключевые идеи:

- **Категории** - включаете только то, что отлаживаете. Не нужен AI-лог, когда ищете баг в физике.
- **Временные метки** - видно, когда именно произошло событие.
- **Условные логи** - пишутся только при проблемах. "Логируй, когда FPS падает ниже 30" полезнее, чем "логируй каждый кадр".

Собираем всё вместе: консоль разработчика

Все инструменты выше можно объединить в одну панель, переключаемую по клавишам:

Клавиша	Инструмент
F1	Визуальная отладка: хитбоксы, состояния ИИ, пути
F2	Замедление (×0.1)
F3	Нормальная скорость
F4	Пауза
F5	Шаг на один кадр (при паузе)
F6	FPS-график
F7	Профайлер по системам
~	Текстовый лог в углу экрана

Это ваша **консоль разработчика**. Профессиональные движки (Unity, Unreal) уже предоставляют эти и другие инструменты. Вы строите свои - и понимаете, как они работают изнутри.

Не удаляйте инструменты отладки после того, как нашли баг. Оставьте их за `if debug_enabled`. Следующий баг не заставит себя ждать.

Нюансы

Инструменты отладки - это тоже код, который работает каждый кадр. Визуальная отладка с сотнями `draw_circle` сама может просадить FPS. Профайлер с `perf_counter()` на каждую систему добавляет накладные расходы. Грубо говоря, эффект наблюдателя - наблюдатель влияет на наблюдаемое.

На практике это редко проблема - отладочный `draw` дешевле игрового рендера. Но если вы видите странные цифры в профайлере, попробуйте отключить визуальную отладку и замерить снова.

И да. Всё, что я тут написали - это DIY-инструменты. В Unity есть Profiler, Frame Debugger, Physics Debugger из коробки. В Unreal - Stat commands, Insights. Смысл строить своё - понять, как это работает изнутри, а не заменить профессиональные инструменты.

Итого

Что	Где подробнее
deltaTime и игровой цикл	Основной цикл
Хитбоксы и коллизии	Коллизии
Состояния ИИ, пути, радиусы	ИИ врагов, деревья поведений
Поиск путей для визуализации	Уровни и поиск путей
Экранные координаты (camera.apply)	Камера
ECS и посистемное обновление	ECS и EC
Типичные ошибки с производительностью	Типичные ошибки
Пространственные оптимизации	Пространственные структуры

Дальше можно копать в сторону assertions (проверки инвариантов - "здоровье не может быть отрицательным"), систем записи и воспроизведения (replay), консольных команд прямо в игре (спавн врагов, телепортация, переключение уровней) - и т.д.

Powered by Wiki.js